

I. Objectifs

Dans le cadre de cet atelier, nous allons aborder les notions basiques de MCP spécifiquement l'utilisation des Tools avec un mode de transport local.

II. Pré-requis

Méthode recommandée : avec uv

- *Installation de uv* : <https://pydevtools.com/handbook/how-to/how-to-install-uv/>

1. Ouvrir le terminal

```
uv venv
```

2. Activez l'environnement

```
./.venv/Scripts/activate
```

3. Installez les dépendances

```
uv add "mcp[cli]"
```

Option : avec pip

1. Ouvrir le terminal

```
python -m venv .venv
```

2. Activez l'environnement

```
./.venv/Scripts/activate
```

3. Installez les dépendances

```
pip install "mcp[cli]"
```

III. Exemple Simple

1. Instanciation

```
mcp = FastMCP()
```

⇒ Vous pouvez ajouter un nom à votre MCP

```
mcp = FastMCP(name="MyFirstMCP")
```

2. Créer un premier outil

```
@mcp.tool()
def say_hello() -> str:
    """Retourne un message de bienvenue"""
    return "Hello World ! "
```

3. Lancement du serveur

```
if __name__ == "__main__":
    mcp.run()
```

⇒ Vous pouvez préciser le type de transport dans ce cas « STDIO »

```
mcp.run(transport="stdio")
```

Le code complet est comme suit

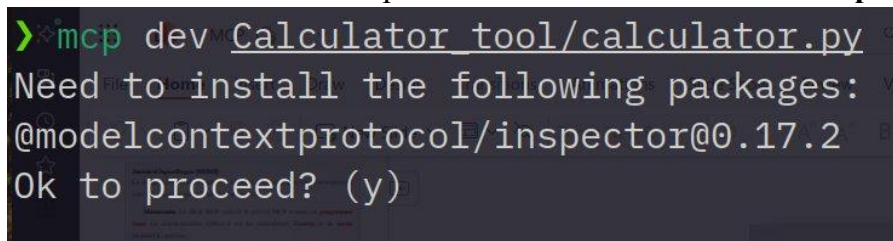
```
from mcp.server.fastmcp import FastMCP

# Create an instance of FastMCP
# All parameters are optional
mcp = FastMCP(
    name="MyCalculator"
)

@mcp.tool()
def say_hello() -> str:
    """Retourne un message de bienvenue"""
    return "Hello World ! "

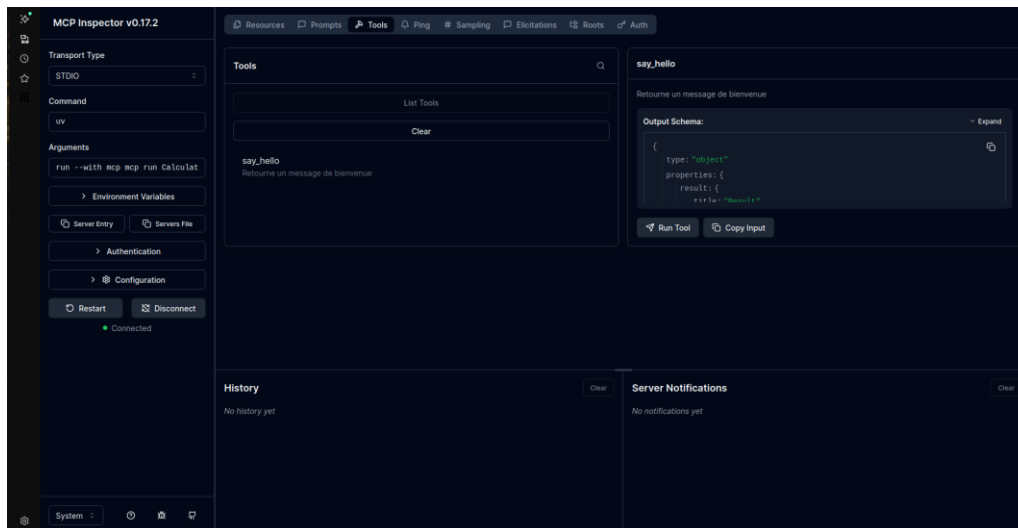
if __name__ == "__main__":
    mcp.run(transport="stdio")
    # You can change transport to "sse" if needed
```

4. Lancer l'exécution de l'inspecteur en utilisant la commande **mcp dev [file_name]**



```
> mcp dev Calculator_tool/calculator.py
Need to install the following packages:
@modelcontextprotocol/inspector@0.17.2
Ok to proceed? (y)
```

Le navigateur va se lancer automatiquement pour ouvrir l'inspecteur.



IV. Exemple Outil

1. Modifier l'outil « say_hello »

```
@mcp.tool()
def say_hello(name: str) -> str:
    """Returns a greeting message."""
    return f'Hello, {name}!'
```

2. Tester l'outil au niveau de l'inspecteur.

V. Tester l'outil avec GitHub Copilot

1. Configurer le fichier *mcp.json*

Le fichier *mcp.json* est le fichier de configuration principal pour intégrer des serveurs MCP (Model Context Protocol) avec GitHub Copilot en mode Agent dans Visual Studio Code (VS Code). Il permet de définir des serveurs locaux (stdio) ou distants (SSE/HTTP) pour étendre les capacités de Copilot.

- Configuration locale

Créez ou trouvez le fichier à la racine de votre projet dans le dossier *.vscode/mcp.json*

ou

Appuyez sur **Ctrl+Shift+P** puis tapez et sélectionnez « **MCP : Open Workspace Configuration** » ⇒ Cela va ouvrir directement le fichier *.vscode/mcp.json*

- Configuration globale

Appuyez sur **Ctrl+Shift+P** puis tapez et sélectionnez « **MCP : Open User Configuration** »

⇒ cela ouvre un mcp.json dans les paramètres utilisateur

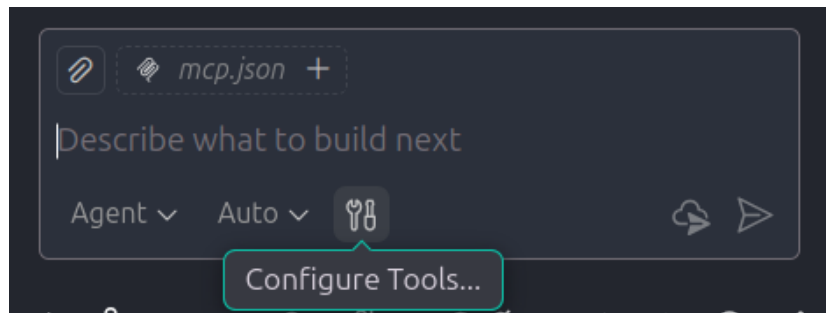
Ajoutez la configuration du serveur au fichier mcp.json

```
{
  "servers": {
    "calculator": {
      "type": "stdio",
      "command": "uv",
      "args": [
        "run",
        "Calculator_tool/calculator.py"
      ],
    }
  }
}
```

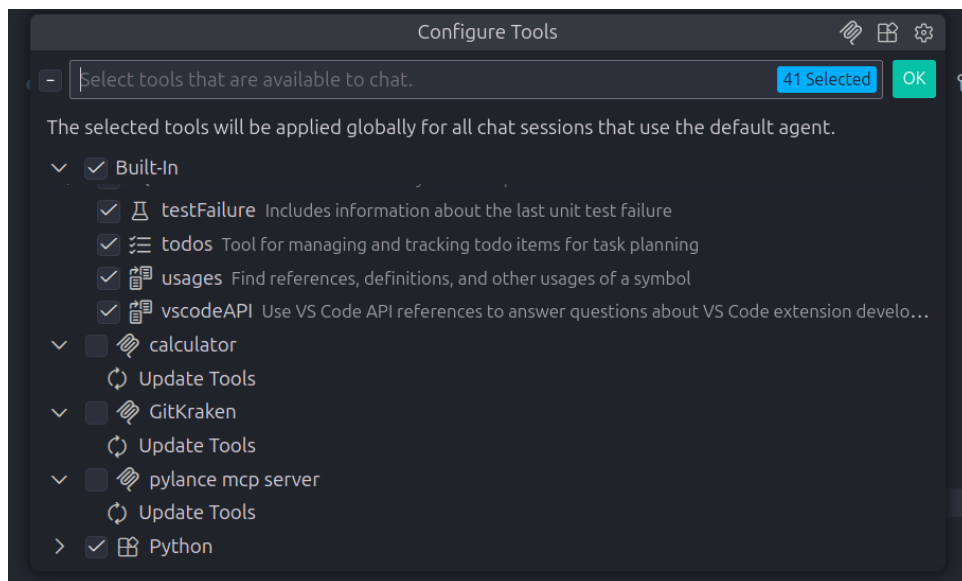
Code plus complet, si pip est utilisé

```
{
  "servers": {
    "calculator": {
      "type": "stdio",
      "command": "python",
      "args": [
        "-m",
        "uv",
        "run",
        "${workspaceFolder}/Calculator_tool/calculator.py"
      ],
    },
    "fallback": {
      "command": "python",
      "args": [
        "${workspaceFolder}/Calculator_tool/calculator.py"
      ]
    }
  }
}
```

Cliquez sur l'icône de configuration des outils dans la fenêtre de copilote



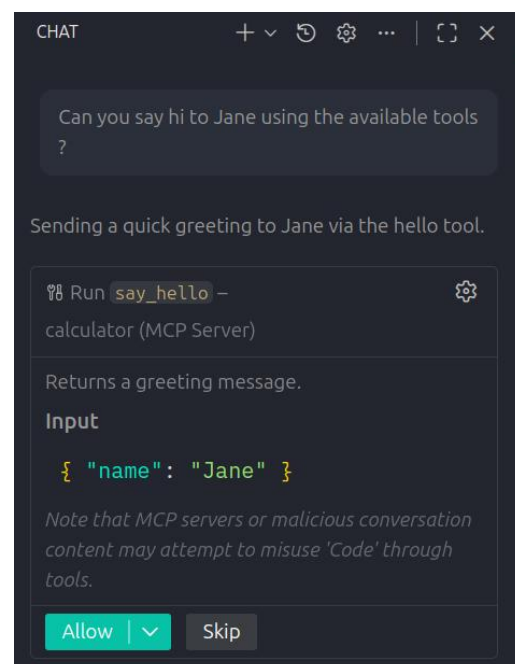
L'utilitaire s'ouvre et vous allez retrouver tous les outils ajoutés au niveau de mcp.json.



En utilisant un prompt tel que montré dans la figure suivante, l'agent détecte la fonction qui lui est disponible et il essaye de l'utiliser.



5



VI. Exemple Calculator

1. Ajoutez vos propres fonctions mathématiques : ajout, soustraction, multiplication et division.
2. Testez ces nouvelles fonctions en utilisant l'inspecteur et en utilisant le mode agent de GitHub Copilot.